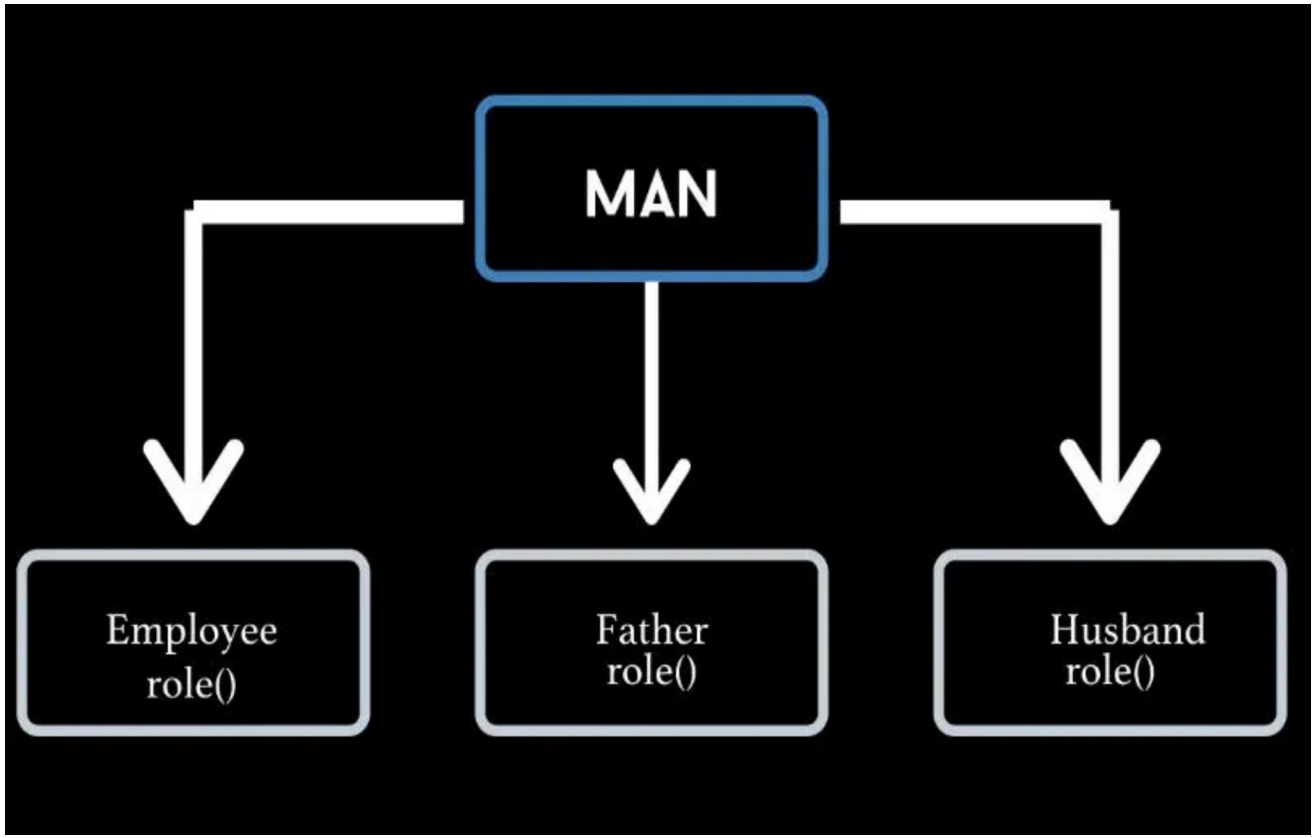
Fundamental Programming in JAVA

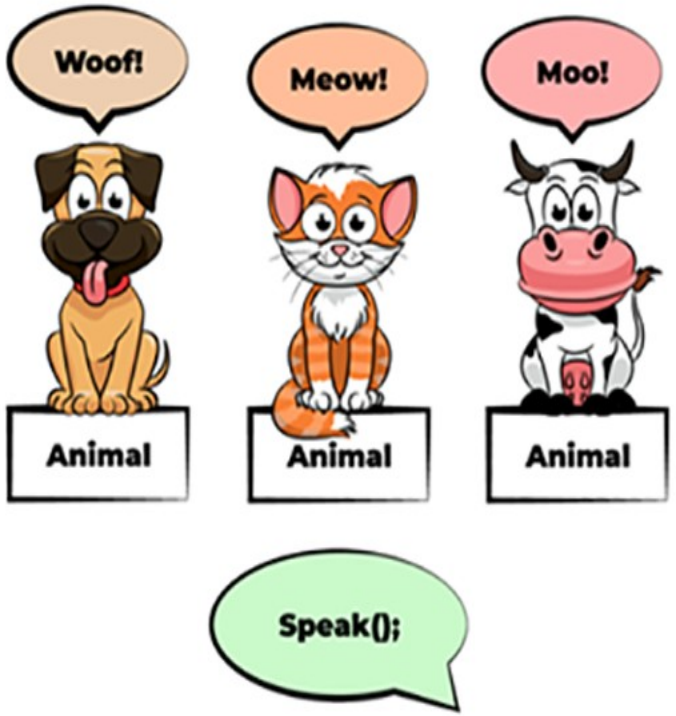
POLYMORPHISM

Objectives

- To discover polymorphism.
- To distinguish between static and dynamic binding.
- To describe casting and explain why explicit casting (down casting) is necessary.
- To demonstrate Polymorphism and casting using an example
- To explore the **equals** method in the **Object** class.

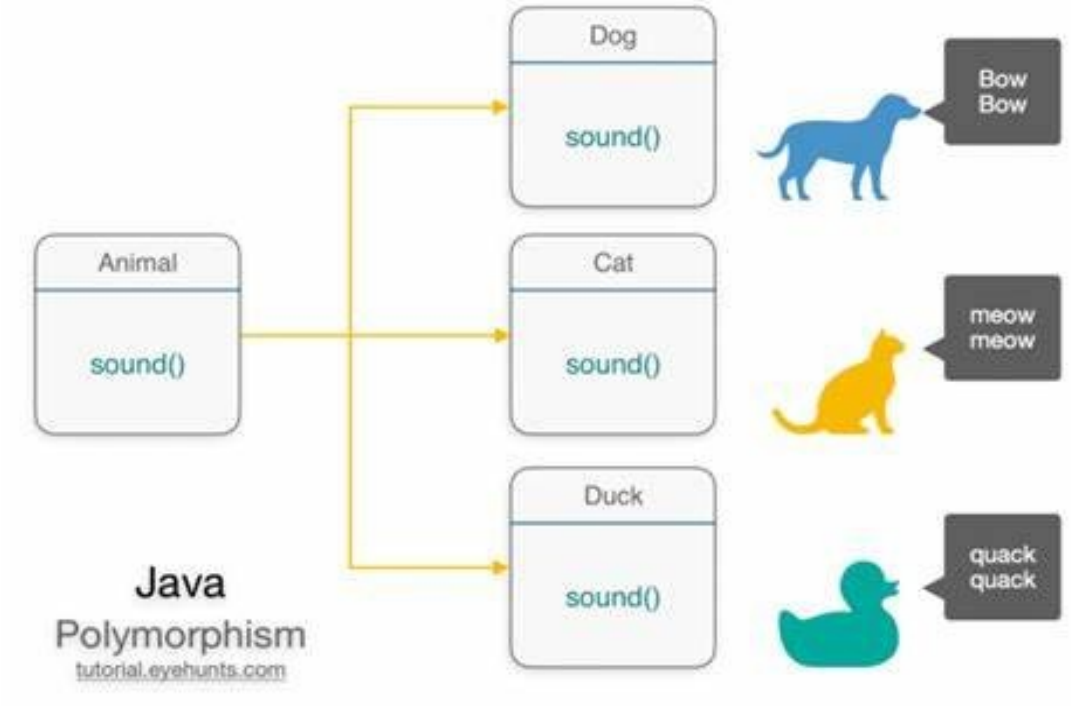
Polymorphism





Polymorphism

- Polymorphism means "many forms“. Objects are designed to share behaviors and they can take on more than one form.

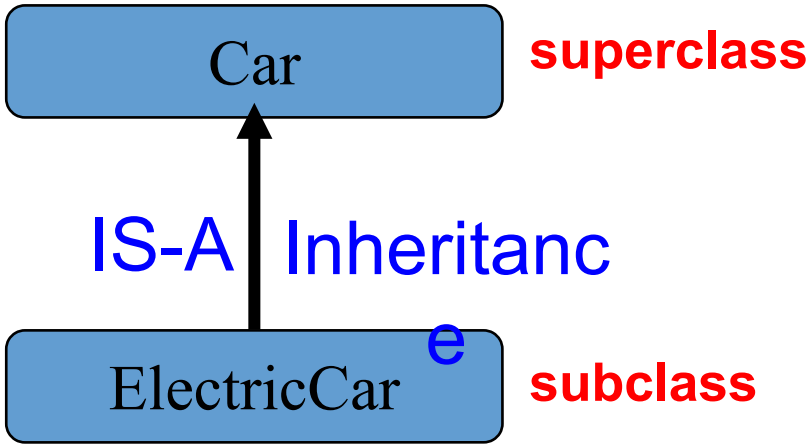


Polymorphism in OOP

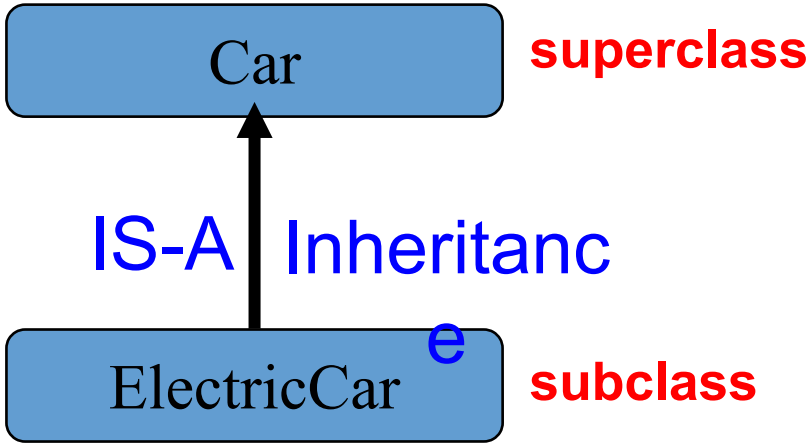
- Polymorphism: Ability for the same code to be used with different types of objects and behave differently with each.
- Inheritance allows you to define a base class and derive classes from the base class.
- Polymorphism allows you to make changes in the method definition for the derived classes and have those changes apply to methods written in the base class.

Polymorphism in OOP

- Polymorphism means that a variable of a *supertype* can refer to a *subtype* object.
- A class defines a type. A type defined by a subclass is called a *subtype*, and a type defined by its superclass is called a *supertype*. Therefore, you can say that ElectircCar is a *subtype* of Car and Car is a *supertype* for ElectircCar.



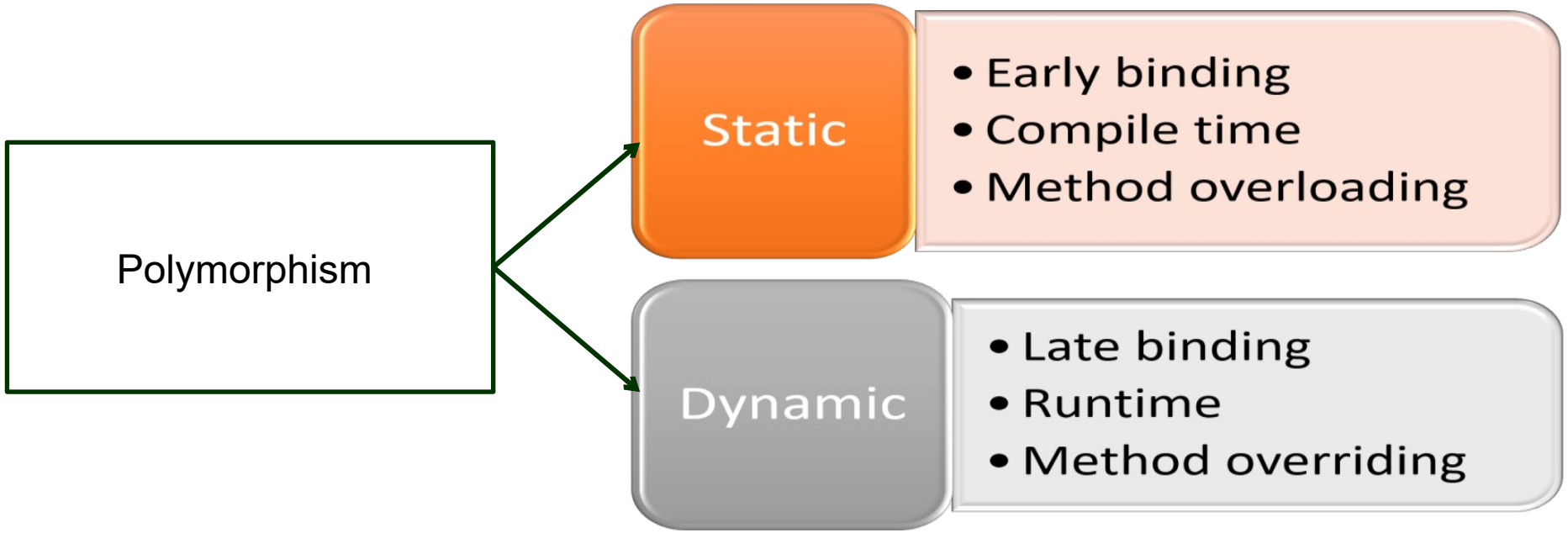
Polymorphism in OOP



<u>Polymorphism:</u> Reference of *supertype* refer to object of *subtype*

```
Car yourCar = new ElectricCar();
```

Types of Polymorphism



Binding

- Binding is what happens when a method invocation is bound to an implementation
  - Involves lookup of the method in the class, or one or its parents.
  - Both method names and parameters are checked.

- Binding can happen at two different times
  - **Compile time == static binding**
  - **Run time == dynamic binding**

Static Binding

- References have a type
  - i.e. they refer to instances of a particular Java class
- Objects have a type
  - i.e. they are instances of a particular Java class
  - They are also instances of their super-class
  - Inheritance describes an IS-A relationship
  - e.g. **an ElectricCar is a Car**

Static Binding

- Static binding done by the compiler
  - When it can determine the type of an object
- Method calls are bound to their implementation immediately

Static Binding Example 1

- Method Overloading

```java
public class MyClass {
    public static void myMethod(int num) {
        System.out.println("The number is " + num);
    }

    public static void myMethod(String str) {
        System.out.println("The string is " + str);
    }

    public static void main(String[] args) {
        MyClass.myMethod(123); // Static binding
        MyClass.myMethod("Hello"); // Static binding
    }
}
```

Static Binding Example 2

```
//class definition
public class ElectricCar {
    public void desBatterty()
     {…}
}//usage statically bound
ElectricCar car = new ElectricCar();
car.desBatterty();
```

Static Binding Example 3

```
public class Car {
    public void start() {…}
    public void stop() {…}
}
public class ElectricCar extends Car() {
   //overridden
   public void start() {…}
   public void desBatterty() {…}
}
```

```
 //reference is to subclass
  //usage still statically bound
ElectricCar myCar = new ElectricCar();
myCar.start();
```

Dynamic Binding

- Java performs *dynamic binding* or *late binding* when a variable contains a polymorphic reference.
- The Java Virtual Machine determines at runtime which method to call, depending on the type of object that the variable references.
- Achieved at runtime
  - When the class of an object cannot be determined at compile time
  - Means the JVM (not the compiler) must bind a method call to its implementation
- Instances of a *subclass* can be treated as if they were an instance of the *superclass*
  - Therefore the compiler doesn't know its type, just its *supertype*.

Dynamic Binding Example 1

```
public class Car {
    public void start() {…}
    public void stop() {…}
}public class ElectricCar extends Car() {
   //overridden
   public void start() {…}
   public void desBatterty() {…}
}
```

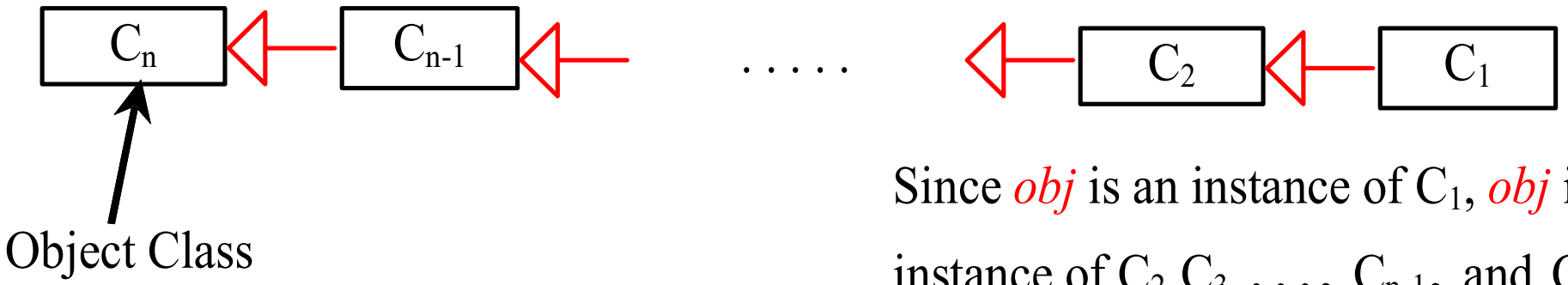
```
//reference is to superclass
// object is subclass
//method is dynamically bound to
ElectricCar start method

Car yourCar = new ElectricCar();
yourCar.start();
```

Dynamic Binding

Dynamic binding works as follows:

- Suppose an object *obj* is an instance of classes C_1, C_2, ..., C_{n-1}, and C_n, where C_1 is a subclass of C_2, C_2 is a subclass of C_3, ..., and C_{n-1} is a subclass of C_n.
- That is, C_n is the most general class (Object Class), and C_1 is the most specific class.



Since *obj* is an instance of C_1, *obj* is also an instance of C_2, C_3, . . ., C_{n-1}, and C_n

Dynamic Binding

Dynamic binding works as follows:

- In Java, C_n is the Object class. If *obj* invokes a method *p*, the JVM searches the implementation for the method *p* in C_1, C_2, ..., C_{n-1} and C_n, in this order, until it is found.
- Once an implementation is found, the search stops and the first-found implementation is invoked.

Dynamic Binding Example 2

```
//remember all classes derive from Object Class
  Object Obj = new ElectricCar();

   Obj.toString();
```

Since *obj* is an instance of ElectricCar, *obj* is also an instance of Car, and Object Class

Benefits of Polymorphism

- Generic Programming: Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as generic programming.
- If a method's parameter type is a superclass (e.g., Object), you may pass an object to this method of any of the parameter's subclasses (e.g., Student or String).
- When an object (e.g., a Student object or a String object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., toString) is determined dynamically.

Generic Programming (Generic Method)

```java
public class DynamicBindingDemo {
  public static void main(String[] args)
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }
  public static void m(Object x) {
    System.out.println(x.toString());
  }
}
```

```java
class Person extends Object {
  @Override
  public String toString() {
    return "Person";
  }
```

```java
class Student extends Person {
  @Override
  public String toString() {
    return "Student";
  }
}
```

```java
class GraduateStudent extends Student {
}
```

Output

```
run:
Student
Student
Person
java.lang.Object@15db9742
BUILD SUCCESSFUL (total time: 0 seconds)
```

Generic Programming (Generic Array)

```java
public class Box {
  private int l,w,h;
  public Box(int a, int b, int c) {
    l=a;  w=b;  h=c;      }

  public void info()        {
    System.out.println("Box:  " + this + "\n");  }

  public String toString(){
    return "length= "+l+" width= "+w+" hight= "+ h; }
}
```

```java
public class WoodenBox extends Box{
  private String level;
  public WoodenBox(int a, int b , int c, String d) {
    super(a,b,c);
    level=d;                    }

  public void info()        {
    System.out.println("Wooden Box: " + this + "\n");  }

  public String toString(){
    return super.toString()+ "   Level= " + level;    }
}
```

Generic Programming (Generic Array)

```java
public class SteelBox extends Box {
  private int weight;

  public SteelBox(int a, int b , int c, int d)
    {   super(a,b,c);
        weight=d;                        }

  public void info()    {
    System.out.println("Steel Box:   " + this + "\n");    }

  public String toString(){
    return super.toString()+ "    Wight= "+ weight;      }
}
```

```java
public class LargeWoodenBox extends WoodenBox{
 public LargeWoodenBox(int a, int b, int c, String d )    {
        super(a, b, c, d);       }

 public void info()      {
    System.out.println("Large Wooden Box:   "
                        + super.toString() + "\n");   }
}
```

```java
public class Demo {
  public static void main ( String arg[ ] )    {
    Box b[]=new Box[6];
        b[0]=new Box(1, 2, 3);
        b[1]=new WoodenBox(2, 3, 4, "good");
        b[2]=new SteelBox(4, 6, 5, 7);
        b[3]=new LargeWoodenBox(6, 5, 4, "bad");
        b[4]=new WoodenBox(9, 8, 7, "very good");
        b[5]=new Box(12, 13, 14);

    for(int i=0; i<b.length; i++)
        b[i].info();
}   }
```

Checking an Object Type

- Its possible to check the actual type of an object
  - May want to check the real type, if we've only got a reference to a superclass
- Use the **`instanceof` operator** to test whether an object is an instance of a class.

Checking an Object Type Example

```
public class Car {

    // Source Code

}
```

```
public class ElectricCar extends Car{

    //Source Code

}
```

Checking an Object Type Example

```java
public class test {
    public static void main(String[] args) {
Car myCar = new Car();
ElectricCar yourCar = new ElectricCar();
Car herCar = new ElectricCar();

System.out.println("IS myCar instanceof Car: " +(myCar instanceof Car));

System.out.println("IS yourCar instanceof ElectricCar : " +
        (yourCar instanceof ElectricCar));

System.out.println("IS yourCar instanceof Car : " +
        (yourCar instanceof Car));

System.out.println("IS herCar instanceof ElectricCar : "
        + (herCar instanceof Car));

System.out.println("IS herCar instanceof ElectricCar : "
        + (herCar instanceof ElectricCar));
```

```
run:
IS myCar instanceof Car: true
IS yourCar instanceof ElectricCar : true
IS yourCar instanceof Car : true
IS herCar instanceof ElectricCar : true
IS herCar instanceof ElectricCar : true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Implicit Casting

- The casting operator already used to convert variables of one primitive type to another.
- Casting can also be used to convert an object of one class type to another within an inheritance hierarchy.

```
Object obj = new Student(); // Implicit casting
```

Implicit Casting

- The casting operator already used to convert variables of one primitive type to another.
- Casting can also be used to convert an object of one class type to another within an inheritance hierarchy.

```
Object obj = new Student(); // Implicit casting
```

- The statement Object obj = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object

Explicit casting

- Suppose you want to assign the object reference ***obj*** to a variable of the Student type using the following statement:

```
Student b = obj;
```

→ A compile error would occur.

Why does the statement **Object obj = new Student()** work

and the statement **Student b = obj** doesn't?

This is because a Student object is always an instance of Object, but an Object is not necessarily an instance of Student. Even though you can see that ***obj*** is really a Student object, the compiler is not so clever to know it.

Explicit casting

- To tell the compiler that *obj* is a Student object, use an explicit casting. The syntax is similar to the one used for casting among primitive data types. Enclose the target object type in parentheses and place it before the object to be cast, as follows:

```
Student b = (Student)obj; // Explicit casting
                          // Down casting
```

Explicit casting

- If we know the type, we can then “cast” the object

```
Car myCar = new ElectricCar();

   if (mayCar instanceof ElectricCar) {
      ElectricCar xCar = (ElectricCar)myCar;
      }
```

- If the object isn’t of that type, then an exception will be thrown
  - Good idea to always check before casting, unless you’re absolutely sure!

Example: Demonstrating Polymorphism and Casting

- This example creates two geometric objects: a circle, and a rectangle, invokes the displayGeometricObject method to display the objects. The displayGeometricObject displays the area and diameter if the object is a circle, and displays area if the object is a rectangle.

```java
package polymorphismexample;
public class SimpleGeometricObject {
  private String color = "white";
  private boolean filled;
  private java.util.Date dateCreated;
  /** Construct a default geometric object */
  public SimpleGeometricObject() {
    dateCreated = new java.util.Date();
  }
  /** Construct a geometric object with the specified color
   *  and filled value */
  public SimpleGeometricObject(String color, boolean filled) {
    dateCreated = new java.util.Date();
    this.color = color;
    this.filled = filled;
  }
  /** Return color */
  public String getColor() {
    return color;
  }
```

Example: Demonstrating Polymorphism and Casting(cont)

```java
21    /** Set a new color */
      public void setColor(String color) {
23      this.color = color;
24    }
25    /** Return filled. Since filled is boolean,
26       its get method is named isFilled */
      public boolean isFilled() {
28      return filled;
29    }
30    /** Set a new filled */
      public void setFilled(boolean filled) {
32      this.filled = filled;
33    }
34    /** Get dateCreated */
      public java.util.Date getDateCreated() {
36      return dateCreated;
37    }
38    /** Return a string representation of this object */
      public String toString() {
40      return "created on " + dateCreated + "\ncolor: " + color +
41        " and filled: " + filled;
42    }
43  }
```

Example: Demonstrating Polymorphism and Casting(cont)

```java
package polymorphismexample;
public class CircleFromSimpleGeometricObject
    extends SimpleGeometricObject {
  private double radius;
  public CircleFromSimpleGeometricObject() {
  }
  public CircleFromSimpleGeometricObject(double radius) {
    this.radius = radius;
  }
  public CircleFromSimpleGeometricObject(double radius,
      String color, boolean filled) {
    this.radius = radius;
    setColor(color);
    setFilled(filled);
  }
  /** Return radius */
  public double getRadius() {
    return radius;
  }
```

```java
  /** Set a new radius */
  public void setRadius(double radius) {
    this.radius = radius;
  }
  /** Return area */
  public double getArea() {
    return radius * radius * Math.PI;
  }

  /** Return diameter */
  public double getDiameter() {
    return 2 * radius;
  }
   /** Return perimeter */
  public double getPerimeter() {
    return 2 * radius * Math.PI;
  }

  /* Print the circle info */
  public void printCircle() {
    System.out.println("The circle is created " +
   getDateCreated() + "and the radius is " + radius);
  }
}
```

Example: Demonstrating Polymorphism and Casting(cont)

```java
package polymorphismexample;
public class RectangleFromSimpleGeometricObject
    extends SimpleGeometricObject {
  private double width;
  private double height;
  public RectangleFromSimpleGeometricObject() {
  }
  public RectangleFromSimpleGeometricObject(
      double width, double height) {
    this.width = width;
    this.height = height;
  }
  public RectangleFromSimpleGeometricObject(
      double width, double height, String color,
      boolean filled) {
    this.width = width;
    this.height = height;
    setColor(color);
    setFilled(filled);
  }
```

```java
  /** Return width */
  public double getWidth() {
    return width;
  }
  /** Set a new width */
  public void setWidth(double width) {
    this.width = width;
  }

  /** Return height */
  public double getHeight() {
    return height;
  }
  /** Set a new height */
  public void setHeight(double height) {
    this.height = height;
  }

  /** Return area */
  public double getArea() {
    return width * height;
  }
  /** Return perimeter */
  public double getPerimeter() {
    return 2 * (width + height);
  }
}
```

Example: Demonstrating Polymorphism and Casting(cont)

```
1
2   package polymorphismexample;
3   public class CastingDemo {
4     /** Main method */
5     public static void main(String[] args) {
6       // Create and initialize two objects
7       Object object1 = new CircleFromSimpleGeometricObject(1);
8       Object object2 = new RectangleFromSimpleGeometricObject(1, 1);
9       // Display circle and rectangle
10      displayObject(object1);
11      displayObject(object2);
12    }
```

Example: Demonstrating Polymorphism and Casting(cont)

```
13      /** A method for displaying an object */
        public static void displayObject(Object object) {
15        if (object instanceof CircleFromSimpleGeometricObject) {
16          System.out.println("The circle area is " +
17            ((CircleFromSimpleGeometricObject)object).getArea());
18          System.out.println("The circle diameter is " +
19            ((CircleFromSimpleGeometricObject)object).getDiameter());
20        }
21        else if (object instanceof
22          RectangleFromSimpleGeometricObject) {
23          System.out.println("The rectangle area is " +
24            ((RectangleFromSimpleGeometricObject)object).getArea());
25        }
26      }
27    }
```

The `equals` Method

- The `equals()` method compares the contents of two objects. The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {
  return this == obj;
}
```

The `equals` Method

- For example, the equals method is overridden in the Circle class.

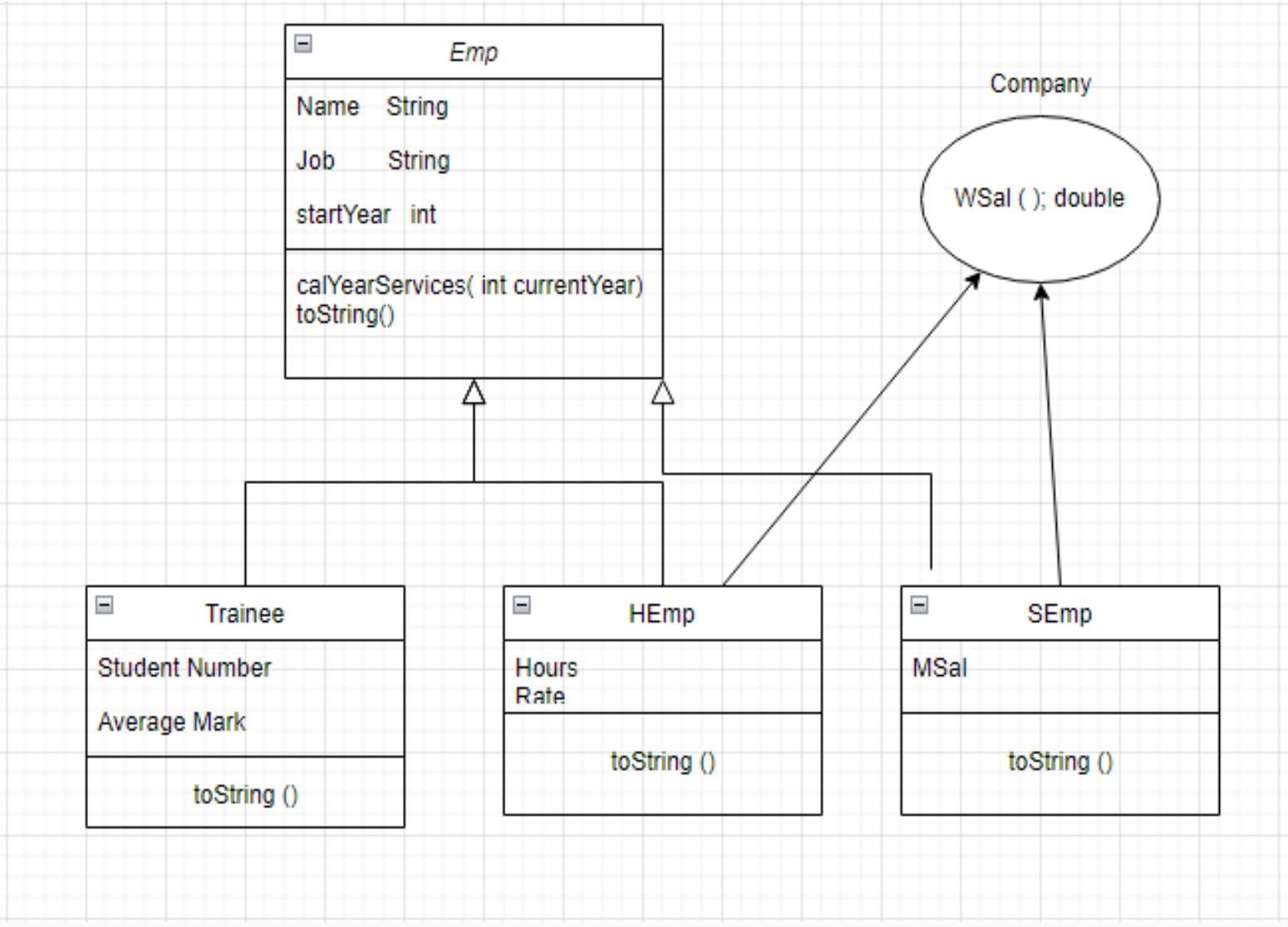
```
public boolean equals(Object o) {
  if (o instanceof Circle) {
    return radius == ((Circle)o).radius;
  }
  else
    return false;
}
```

NOTE

- The **==** comparison operator is used for comparing two primitive data type values or for determining whether two objects have the same references. The **equals** method is intended to test whether two objects have the same contents, provided that the method is modified in the defining class of the objects. **The == operator is stronger than the equals method, in that the == operator checks whether the two reference variables refer to the same object.**

Example

Example: Going back to the company that has two types of employees, the first type gets a monthly salary (**SEmp**), and the other gets an hourly-based salary (**HEmp**). This company also employs (**Trainee**) who work for experience only and receive no pay from the employer.



Example

- **In the main class**, define an array that can hold any type of employee and go through them to call each method in every class (i.e. calYearServices (int currentYear), toString(), and Wsal())

Now, Can we define one array that can hold objects from *SEmp* and *HEmp* at the same time? Yes. Define an array of Emp:

```
1   public class test {
2       public static void main(String[] args) {
3           Emp[] obj = new Emp[10];
4           obj [0]=new Emp();
5           obj[1]= new SEmp("Ahmad", "programmer", 2012, 800);
6           obj[2]=new HEmp("sami", "accountant", 2015, 8, 10);
7           obj[3]=new Trainee("Marwa","programmer", 2023, 8);
8       }
9   }
```

Example(cont)

- **You have undoubtedly noticed** that some methods, like calYearServices (int currentYear) and toString()), exist in both the superclass and the subclasses, while others, like Wsal(), only exist in the subclasses. **Let's** demonstrate how these two situations should be handled when the loop statement is used.

First situation (Method exist in both the superclass and the subclasses):

```
1  public class test { public static void main(String[] args) {
2          Emp[] obj = new Emp[10];
3          obj [0]=new Emp();
4          obj[1]= new SEmp("Ahmad", "programmer", 2012, 800);
5          obj[2]=new HEmp("sami", "accountant", 2015, 8, 10);
6          obj[3]=new Trainee("Marwa","programmer", 2023, 8);
7
8          for(int i=0; i<4; i++){
9                  System.out.println(obj[i].toString()); }
10     }}
```

Note that The statement that is shown in line **9 will not** generate a compilation time error. That is because **obj** is of the Emp type, and Emp defines the toString method.

Example(cont)

Now let's call `calYearServices(int currentyear)` for each method in every class.

```
1  public class test { public static void main(String[] args) {
2          Emp[] obj = new Emp[10];
3          obj[0] = new Emp();
4          obj[1] = new SEmp("Ahmad", "programmer", 2012, 800);
5          obj[2] = new HEmp("sami", "accountant", 2015, 8, 10);
6          obj[3] = new Trainee("Marwa", "programmer", 2023, 8);
7          for (int i = 0; i < 4; i++) {
8              System.out.println("No. of Year Services for "+obj[i].name+" is:"
9
10                     +obj[i].calYearServices(2023));
11         }
12     }}
```

Example(cont)

Second situation (Method only exist in the subclasses):

```
1  public class test { public static void main(String[] args) {
2          Emp[] obj = new Emp[10];
3          obj[0] = new Emp();
4          obj[1] = new SEmp("Ahmad", "programmer", 2012, 800);
5          obj[2] = new HEmp("sami", "accountant", 2015, 8, 10);
6          obj[3] = new Trainee("Marwa", "programmer", 2023, 8);
7          for (int i = 0; i < 4; i++) {
8
9           System.out.println("Wsal of "+obj[i].name+" is:" + obj[i].Wsal());
10
11         }
12     }}
```

Note that The statement that is shown in line **9 will** (); will generate compilation error, since **obj** is of type Emp, and Emp didn't define this method. We can't compute the total salaries directly, as we did with abstract classes; this is because WSal() is not defined in the superclass (the array type). **Okay**, we should use **casting** in order to **<u>bypass the error</u>**, but should we cast obj[i] to SEmp or Hemp?See the next slide to determine to whom we should cast obj [i])

Example(cont)

```java
public class test { public static void main(String[] args) {
        Emp[] obj = new Emp[10];
        obj[0] = new Emp();
        obj[1] = new SEmp("Ahmad", "programmer", 2012, 800);
        obj[2] = new HEmp("sami", "accountant", 2015, 8, 10);
        obj[3] = new Trainee("Marwa", "programmer", 2023, 8);
        for (int i = 0; i < 4; i++) {

      if(obj[i]instanceof SEmp){
      System.out.println("Wsal of "+obj[i].name+" is:" + ((SEmp)obj[i]).Wsal());

      else if(obj[i]instanceof HEmp){
      System.out.println("Wsal of "+obj[i].name+" is:" + ((HEmp)obj[i]).Wsal())}
        }
    }}
```

In this example, we just check if obj[i] is an instance of a **SEmp** or **HEmp**. Why?

Answer: Obj [i] might also be **Emp** or **Trainee,** isn't it? and you probably know that neither of them defines this method (Wsal).

Example(cont)

```java
public class test {

    public static void main(String[] args) {
        Emp[] obj = new Emp[10];
        obj[0] = new Emp();
        obj[1] = new SEmp("Ahmad", "programmer", 2012, 800);
        obj[2] = new HEmp("sami", "accountant", 2015, 8, 10);
        obj[3] = new Trainee("Marwa", "programmer", 2023, 8);
        for (int i = 0; i < 4; i++) {
            System.out.println(obj[i].toString());
            System.out.println(obj[i].calYearServices(2023));
            if (obj[i] instanceof SEmp) {
                System.out.println(((SEmp) obj[i]).Wsal());
            } else if (obj[i] instanceof HEmp) {
                System.out.println(((HEmp) obj[i]).Wsal());
            }
        }
    }
}
```

References

- Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos: YouTube channel.
- How to install JDK & NetBeans IDE manual.